

数据结构学习报告

成员分工

王枫：代码和学习报告编写

沈子力：演讲，书本文字提取

秦朗：简要排版

查找

一、基于树的查找

线性表查找把记录放在数组或链表中；树表查找则把记录组织成树形结构，并根据关键字大小沿某一条分支继续查找。这样既能保持一定的查找效率，又比有序顺序表更方便插入和删除。

1. 二叉排序树

二叉排序树又称二叉查找树。它或者是一棵空树，或者满足以下性质：

1. 若左子树非空，则左子树上所有结点的关键字均小于根结点关键字。
2. 若右子树非空，则右子树上所有结点的关键字均大于根结点关键字。
3. 左、右子树也分别是二叉排序树。
4. 重要性质：**中序遍历二叉排序树可以得到递增有序序列**。若采用逆中序遍历，也就是先遍历右子树，再访问根结点，最后遍历左子树，则可以得到递减有序序列。

这是一个递归定义。由于左子树小于根、右子树大于根，因此二叉排序树的查找过程很像折半查找：每到一个结点，就根据比较结果只选择一侧子树继续查找。

二叉排序树的结点结构可以写成：

```
typedef struct node {
    KeyType key;
    struct node *lchild, *rchild;
} BSTNode, *BSTree;
```

二叉排序树的查找

二叉排序树的递归查找代码如下：

```

BSTree SearchBST(BSTree bst, KeyType key) {
    if (!bst || bst->key == key)
        return bst; //可合并
    else if (key < bst->key)
        return SearchBST(bst->lchild, key);
    else
        return SearchBST(bst->rchild, key);
}

```

递归写法很贴近定义：空树表示失败；等于根结点则成功；小于根结点去左子树；大于根结点去右子树。

非递归用循环本质相同。

二叉排序树的插入与创建

二叉排序树的插入总是把新结点插到叶子位置。若树为空，新结点成为根；若树非空，则从根开始比较，小于当前结点就进入左子树，大于当前结点就进入右子树，直到找到空位置。

教材中的递归插入代码可以整理为：

```

void InsertBST(BSTree *bst, KeyType key) {
    BSTree s;

    if (*bst == NULL) {
        s = (BSTree)malloc(sizeof(BSTNode));
        s->key = key;
        s->lchild = NULL;
        s->rchild = NULL;
        *bst = s;
    } else if (key < (*bst)->key) {
        InsertBST(&(*bst)->lchild, key);
    } else if (key > (*bst)->key) {
        InsertBST(&(*bst)->rchild, key);
    }
}

```

如果关键字相等，教材代码不再插入。这样可以保证树中关键字不重复。如果实际应用允许重复关键字，就需要约定重复值放在左子树、右子树，或者在结点中增加计数字段。

创建二叉排序树就是从空树开始，按输入顺序逐个插入关键字：

```

void CreateBST(BSTree *bst) {
    KeyType key;

    *bst = NULL;
    scanf("%d", &key);

    while (key != ENDKEY) {
        InsertBST(bst, key);
        scanf("%d", &key);
    }
}

```

这里有一个重要结论：**同一组关键字，如果输入顺序不同，生成的二叉排序树形态可能不同**。例如按接近均匀的顺序插入，树可能比较平衡；如果按递增或递减顺序插入，树会退化成单链表。

注意：二叉排序树的插入只能在叶子节点处修改。假如要修改内部，树的结构很与有可能改变，时间复杂度激增，个人认为是二叉排序树的问题，创建的时候最好是已经排好序的状态

二叉排序树的删除

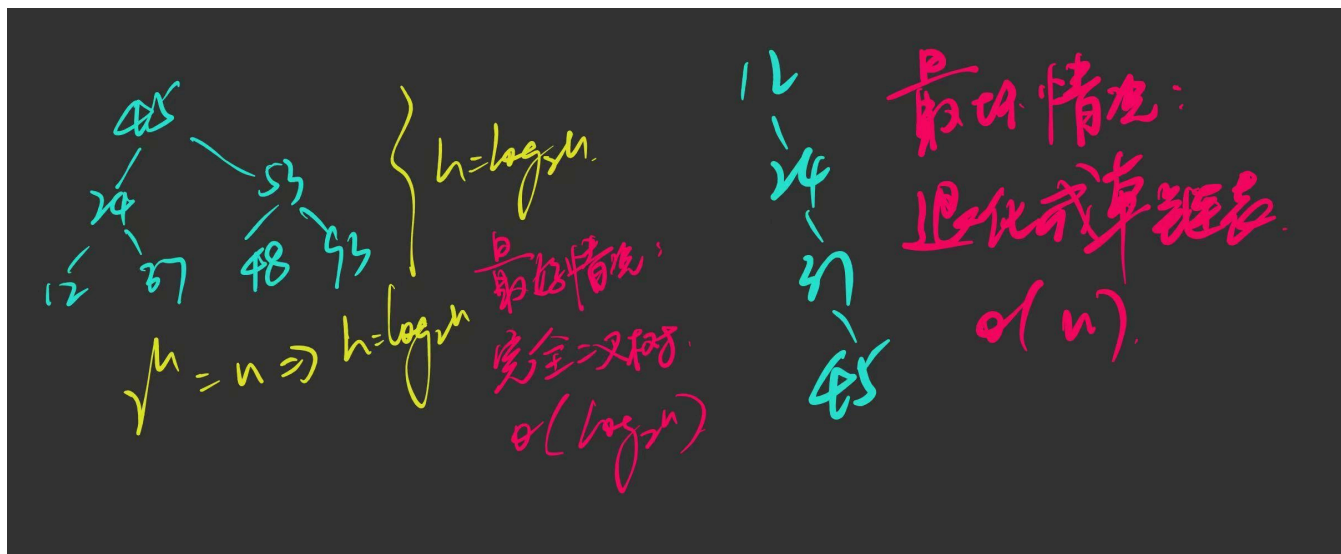
删除二叉排序树中的结点时，不能简单删掉以该结点为根的整棵子树，而要在删除后仍保持二叉排序树性质。设待删结点为 p ，通常分三种情况：

1. p 是叶子结点：直接删除。
2. p 只有一棵子树：让 p 的双亲直接指向这棵子树。
3. p 有两棵子树：用 p 的中序前驱或中序后继替代 p ，再删除那个前驱或后继结点。

用中序前驱处理时，核心思想是：在 p 的左子树中找到最右下的结点 s ，它是小于 p 的最大关键字。用 $s \rightarrow key$ 覆盖 $p \rightarrow key$ 后，再删除原来的 s 。因为 s 最多只有左子树，所以删除 s 会转化为前两种简单情况。

二叉排序树时间复杂度

二叉排序树的查找、插入和删除都与树高有关。若树高为 h ，这些操作的时间复杂度为 $O(h)$ 。最好或较平衡情况下， h 接近 $\log_2 n$ ；最坏情况下，树退化成单链表， $h = n$ ，时间复杂度退化为 $O(n)$ 。



2. 平衡二叉排序树

平衡二叉排序树又称 AVL 树。它首先是一棵二叉排序树，同时还要求任意结点的左子树和右子树高度差的绝对值不超过 1。

结点的平衡因子定义为：

$$bf = \text{左子树高度} - \text{右子树高度}$$

对于平衡二叉排序树，每个结点的平衡因子只能是 -1、0 或 1。引入 AVL 树的目的，是避免普通二叉排序树因插入顺序不佳而退化成单链表，从而提高查找效率。

AVL 树的查找过程与普通二叉排序树完全相同，区别主要出现在插入和删除后。如果一次插入导致某个结点的平衡因子变成 2 或 -2，就要通过旋转恢复平衡。

原则：

- 降低排序树高度
- 保持二叉排序树性质

教材中把失衡情况归纳为四类：

失衡类型	插入位置	调整方法	直观理解
LL 型	A 的左子树的左子树	对 A 做一次右旋	左边太高，向右转
RR 型	A 的右子树的右子树	对 A 做一次左旋	右边太高，向左转
LR 型	A 的左子树的右子树	先对 B 左旋，再对 A 右旋	先拉直，再调整
RL 型	A 的右子树的左子树	先对 B 右旋，再对 A 左旋	先拉直，再调整

其中 A 表示最低层失衡结点，B 是 A 较高一侧的孩子，C 是双旋时参与调整的孙子结点。

核心原则：寻找最低层失衡点，给三个数排序，第二大的点作为根节点，其余两点按照二叉排序树性质作为左右子树

LL 型右旋的核心代码可以理解为：

```
A->lchild = B->rchild;
B->rchild = A;
A->bf = 0;
B->bf = 0;
```

RR 型左旋与 LL 型对称：

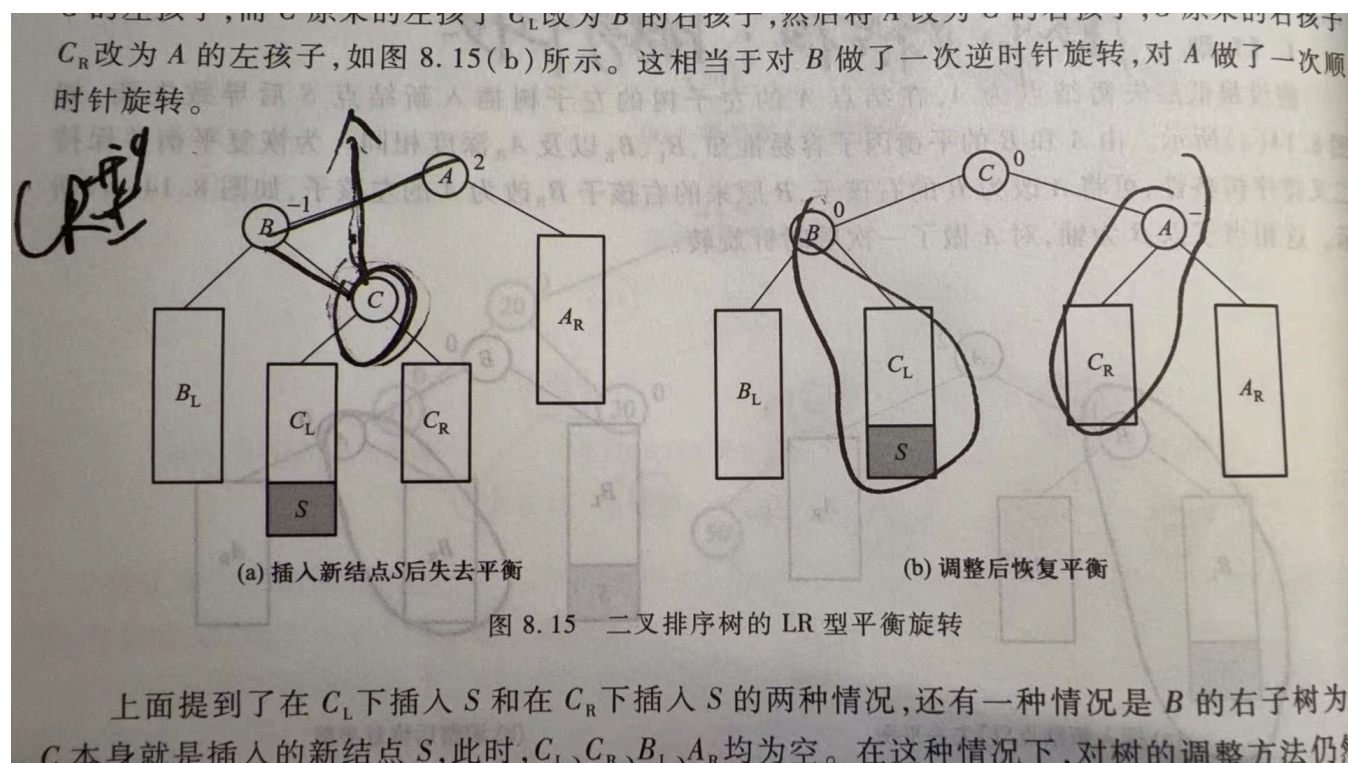
```
A->rchild = B->lchild;
B->lchild = A;
A->bf = 0;
B->bf = 0;
```

LR 型需要先把 B 和 C 调整，再把 A 和 C 调整：

```
B->rchild = C->lchild;
A->lchild = C->rchild;
C->lchild = B;
C->rchild = A;
```

RL 型与 LR 型对称：

```
B->lchild = C->rchild;
A->rchild = C->lchild;
C->lchild = A;
C->rchild = B;
```



旋转的目标不是改变二叉排序树的中序序列，而是在保持中序有序的前提下降低树高。也就是说，旋转前后关键字的大小关系不变，只是树的形态变得更均衡。

AVL 树的插入过程可以概括为：

1. 按二叉排序树的方法查找插入位置。
2. 插入新结点。
3. 从插入位置向上更新平衡因子。
4. 找到最低层失衡结点。
5. 根据 LL、RR、LR、RL 四种类型进行旋转。

由于 AVL 树始终保持高度平衡，它的树高为 $O(\log n)$ ，因此查找、插入和删除的时间复杂度都是 $O(\log n)$ 。不过 AVL 树为了维护平衡，需要额外保存平衡因子，并在插入、删除时进行旋转，所以实现复杂度高于普通二叉排序树。

二、哈希查找

哈希查找又称散列查找、杂凑查找或关键字地址计算法。它的基本思想是：在关键字 k 和存储位置 p 之间建立一个函数关系

$$p = H(k)$$

其中 $H(k)$ 称为哈希函数。创建哈希表时，把关键字为 k 的记录存入地址 $H(k)$ ；查找时，再通过同一个函数计算位置。如果没有冲突，就可以接近常数时间完成查找。

哈希查找要解决两个核心问题：如何构造哈希函数，以及如何处理冲突。

1. 哈希函数的构造

构造哈希函数时，一般希望满足两个原则：第一，函数本身计算简单；第二，计算出的地址分布均匀，尽量减少冲突。

教材中常见的哈希函数构造方法有以下几种。

数字分析法

数字分析法适用于事先知道关键字集合，且关键字位数比哈希地址位数多的情况。通过分析关键字各位的分布情况，选取分布较均匀的若干位作为哈希地址。

例如一组 8 位十进制关键字中，如果第 3、4 位分布比较均匀，就可以取这两位作为地址。若选中的位分布很集中，就会产生大量冲突。

平方取中法

平方取中法先求关键字的平方，再取平方值中间的若干位作为哈希地址。因为平方后的中间几位通常与关键字的多位都有关，所以在无法判断哪些位分布均匀时，这种方法比较常用。

分段叠加法

分段叠加法按哈希地址所需位数，把关键字分成若干段，然后把各段相加，舍弃最高进位后的结果作为哈希地址。常见方式有移位叠加和折叠叠加。

除留余数法

除留余数法是最常见、最实用的方法。设哈希表长为 m ，选择一个不大于 m 的素数 p ，定义

$$H(k) = k \% p$$

例如表长为 13 时，可以取 $p = 13$ ，则关键字 46 的哈希地址为

$$H(46) = 46 \% 13 = 7$$

实际应用中， p 的选择会明显影响冲突情况。一般尽量选素数，并避免与关键字分布规律产生明显关系。

伪随机数法

伪随机数法采用伪随机函数作为哈希函数，即

$$H(key) = random(key)$$

这种方法适合关键字分布不明确的场景，但要保证同一关键字每次计算出的地址一致。

2. 冲突处理方法

不同关键字可能被映射到同一个哈希地址，这种现象称为冲突。发生冲突的关键字称为同义词。由于关键字集合通常远大于哈希表地址空间，冲突一般无法完全避免，只能尽量减少并妥善处理。

开放寻址法

开放定址法的基本思想是：如果初始地址 $H(key)$ 发生冲突，就按照某个增量序列继续寻找下一个地址，直到找到空单元。

通用形式为

$$H_i = (H(key) + d_i) \% m$$

其中 m 为表长， d_i 为第 i 次探测的增量。

线性探测再散列的增量为

$$d_i = 1, 2, 3, \dots, m - 1$$

它的特点是发生冲突后顺序查看下一个单元。优点是只要表未满，就一定能找到空位置；缺点是容易产生聚集现象，也就是一段连续单元被填满后，更多关键字会被挤到同一片区域。

二次探测再散列的增量通常为

$$d_i = 1^2, -1^2, 2^2, -2^2, \dots$$

它可以减少线性探测中的聚集，但不一定在所有情况下都能探测到表中每个位置。

伪随机探测再散列则使用伪随机数序列作为增量。它也可以减少聚集，但实现时要保证探测序列可重复。

再哈希法

再哈希法同时构造多个哈希函数：

$$H_1(key), H_2(key), \dots, H_k(key)$$

当一个哈希函数产生冲突时，就使用下一个哈希函数重新计算地址，直到不再冲突。这种方法不容易产生聚集，但会增加计算时间。

拉链法

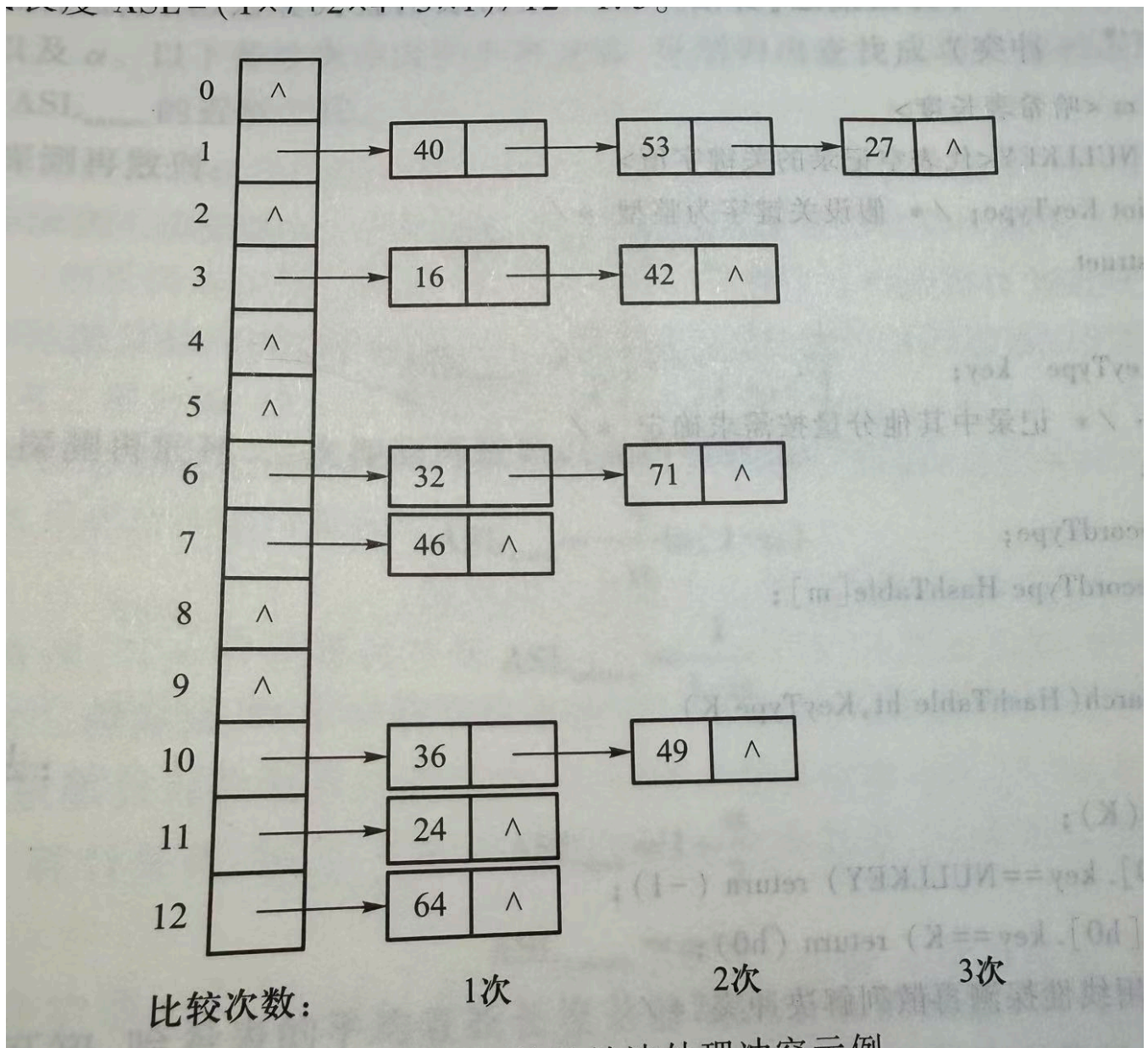
链地址法把所有哈希地址相同的记录放到同一个单链表中。哈希表的每个位置不直接存一个记录，而是存放对应同义词链的头指针。

例如，若 $H(32) = 6$, $H(45) = 6$, $H(58) = 6$ ，则这三个关键字都放在哈希表第 6 个位置对应的链表中。查找时先计算地址，再到该地址对应的链表里顺序查找。

链地址法适合经常插入和删除的情况。因为删除链表中的一个结点比较方便，不会像开放定址法那样影响后续探测路径。

优于开放寻址法

[哈希表 | xiaomao's blog](#)



建立公共溢出区

公共溢出区法把哈希表分成基本表和溢出表。没有冲突的记录放入基本表；凡是发生冲突的记录，统一放入溢出表。查找时先查基本表，再查溢出表。

这种方法思路简单，但如果冲突很多，溢出表会变长，查找效率会下降。

3. 哈希表的查找过程

哈希表的查找过程与建表过程一致。以开放定址法为例，查找关键字 k 的过程是：

1. 计算初始地址 $p = H(k)$ 。
2. 若 $ht[p]$ 为空，则查找失败。
3. 若 $ht[p].key == k$ ，则查找成功。
4. 若 $ht[p]$ 非空但关键字不等，则按冲突处理方法计算下一个地址。
5. 重复上述过程，直到成功、遇到空位置，或者探测完整张表。

用线性探测处理冲突时，查找代码可以写成：

```

#define NULLKEY -1
#define m 16

typedef struct {
    KeyType key;
    OtherType other_data;
} HashTable[m];

int Hash(KeyType key) {
    return key % 13;
}

int HashSearch(HashTable ht, KeyType key) {
    int p = Hash(key);
    int i = 0;

    while (ht[p].key != NULLKEY && ht[p].key != key && i < m) {
        p = (p + 1) % m;      /* 线性探测 */
        i++;
    }

    if (ht[p].key == key)
        return p;
    else
        return -1;
}

```

这里的 `NULLKEY` 表示空单元。若采用开放定址法，删除记录时不能简单把单元置空，否则可能中断后续同义词的查找路径。通常需要设置一个特殊的“已删除”标记。

4. 哈希法性能分析

由于冲突存在，哈希查找并不一定只比较一次。评价哈希表性能时仍然使用平均查找长度。

影响哈希查找性能的主要因素有三个：

1. 哈希函数是否均匀。
2. 冲突处理方法是否合适。
3. 哈希表的装填因子。

装填因子 α 定义为：

$$\alpha = \frac{\text{表中记录数}}{\text{哈希表长度}}$$

装填因子越大，哈希表越满，发生冲突的可能性越高；装填因子越小，冲突越少，但空间浪费越多。

在哈希函数均匀的前提下，哈希表的平均查找长度主要取决于装填因子和冲突处理方法。一个重要结论是：哈希表的平均查找长度主要是 α 的函数，而不是记录总数 n 的函数。因此，即使数据规模很大，只要合理控制装填因子，哈希查找仍可以保持较高效率。

对于具体哈希表，通常可以直接根据每个关键字的查找比较次数计算成功查找的 ASL：

$$ASL_{成功} = \frac{1}{n} \sum_{i=1}^n c_i$$

查找失败的 ASL 则要从每个可能的哈希入口出发，计算从该入口开始直到确定失败需要比较的次数，再求平均。

三、查找方法对比

查找方法	存储结构	是否要求有序	核心思想	平均时间复杂度	插入删除	适用场景
二叉排序树	二叉链表	树中保持有序关系	按大小走左/右子树	平均 $O(\log n)$, 最坏 $O(n)$	较方便	动态查找表
平衡二叉排序树	二叉链表	树中保持有序关系和平衡	查找同 BST, 插入删除后旋转	$O(\log n)$	需要维护平衡	动态查找且要求稳定效率
哈希查找	哈希表	不要求关键字有序	由哈希函数计算地址	平均接近 $O(1)$	视冲突方法而定	快速按关键字访问

从表中可以看出，二叉排序树适合动态查找，但性能受树形影响；平衡二叉排序树通过旋转维护平衡，避免退化；哈希查找平均效率最高，但它牺牲了关键字之间的有序关系，并且需要处理冲突。

四、综合分析

本阶段学习的查找算法可以围绕两条主线理解。

第一条主线是树形查找。二叉排序树把有序表中的大小关系转化成树上的左右分支，查找、插入、删除都沿一条路径进行，因此适合动态查找表。但普通二叉排序树的形态受插入顺序影响，可能退化成单链表。平衡二叉排序树通过平衡因子和旋转控制树高，使最坏情况下也能保持 $O(\log n)$ 的效率。

第二条主线是计算式查找。哈希查找不再通过多次比较逐步缩小范围，而是直接由关键字计算地址。它的理想情况是一次定位，但现实中必须处理冲突。哈希函数越均匀、装填因子越合理、冲突处理越合适，查找效率就越高。

实际选择查找方法时，需要结合表的规模、是否频繁插入删除、是否需要按序遍历等因素。如果需要动态维护有序数据，可以使用二叉排序树或平衡二叉排序树；如果只追求按关键字快速访问，而不关心顺序关系，哈希表通常更有优势。

内部排序

一、概念

排序的目标，是将一组记录按照关键字重新排列，使关键字满足递增或递减关系。若有记录序列 $R_1, R_2, \dots, R_n$ ，对应关键字为 $K_1, K_2, \dots, K_n$ ，排序的结果就是找到一种下标排列 $p_1, p_2, \dots, p_n$ ，使得

$$K_{p_1} \leq K_{p_2} \leq \dots \leq K_{p_n}$$

如果整个排序过程都在内存中完成，称为内部排序；如果数据量过大，排序过程需要借助外部存储设备，称为外部排序。本报告只分析内部排序中已经学习到的插入类排序和交换类排序。

内部排序算法通常围绕两种基本操作展开：一是比较关键字大小，二是移动记录位置。不同算法的差别，主要体现在如何减少比较次数、如何减少移动次数，以及是否需要额外辅助空间。

排序算法还要关注稳定性。若两个记录关键字相等，排序前先出现的记录在排序后仍然排在前面，则该排序方法是稳定的；否则是不稳定的。稳定性在多关键字排序中非常重要。例如先按学号排好，再按成绩排序，如果成绩相同仍要求学号小者在前，就应选择稳定排序。

教材中的代码以记录数组为基础，可以抽象为如下记录结构：

```
typedef int KeyType;

typedef struct {
    KeyType key;
    OtherType other_data;
} RecordType;
```

其中 `key` 是排序关键字，`other_data` 表示记录中除关键字外的其他数据。教材中的数组下标通常从 1 开始，**并用 `r[0]` 作为监视哨或临时存储单元**。

二、插入类排序

插入类排序的基本思想是：维护一个已经有序的子序列，每次从无序部分取出一个元素，将它插入到有序子序列中的正确位置。直接插入排序、折半插入排序和希尔排序都属于这一类。

1. 直接插入排序

直接插入排序将第 i 个记录插入到前面 $i - 1$ 个已经有序的记录中。具体做法是：先保存当前待插入记录，然后从后向前扫描有序区间；凡是关键字大于待插入记录的元素，都向后移动一个位置；当找到不大于待插入记录的位置时，将待插入记录放入空位。

教材中的直接插入排序代码如下：

```
void InsSort(RecordType r[], int n) {
    /* 对记录数组r做直接插入排序，n为数组中待排序记录的数目 */
    int i, j;

    for (i = 2; i <= n; i++) {
        r[0] = r[i];          /* 将待插入记录存放到监视哨r[0]中 */
        j = i - 1;

        while (r[0].key < r[j].key) { /* 寻找插入位置 */
            r[j + 1] = r[j];
            j = j - 1;
        }

        r[j + 1] = r[0];      /* 将待插入记录插入已排序的序列 */
    }
}
```

```
}
```

这段代码的关键是 `r[0]`。它有两个作用：第一，保存待插入记录，避免移动元素时丢失原值；第二，作为监视哨简化边界控制。由于教材数组从 1 开始，当前元素不断向前比较时，`r[0]` 可以防止查找位置时越界。

直接插入排序的最好情况是原序列已经有序，此时每趟只需比较一次，不需要移动大量记录，时间复杂度为 $O(n)$ 。最坏情况是原序列逆序，此时每个元素都要一路前移，时间复杂度为 $O(n^2)$ 。其空间复杂度为 $O(1)$ 。

直接插入排序是**稳定排序**。因为判断条件是 `r[0].key < r[j].key`，当两个关键字相等时，前面的元素不会被后移，后出现的相等元素也就不会插到先出现的相等元素前面。

个人实现中使用的是 0 下标数组和普通变量保存待插入值：

```
int v = A[i];
int j = i - 1;

while (j >= 0 && A[j] > v)
{
    A[j + 1] = A[j];
    j--;
}

A[j + 1] = v;
```

这个写法更符合 C/C++ 常见数组习惯，优点是直观；教材写法的优势是通过监视哨减少边界判断。两种写法的算法思想一致，时间复杂度和空间复杂度也相同。

2. 折半插入排序

折半插入排序是对直接插入排序中“寻找插入位置”这一环节的改进。直接插入排序用顺序查找确定插入位置，而折半插入排序利用前 $i - 1$ 个记录已经有序的特点，用折半查找确定插入位置。

教材中的折半插入排序代码如下：

```
void BinSort(RecordType r[], int n) {
    /* 对记录数组r进行折半插入排序，n为数组的长度 */
    int i, j, low, high, mid;
    RecordType x;

    for (i = 2; i <= n; ++i) {
        x = r[i];
        low = 1;
        high = i - 1;

        while (low <= high) {          /* 确定插入位置，减少比较次数 */
            mid = (low + high) / 2;

            if (x.key < r[mid].key)
                high = mid - 1;
            else
                low = mid + 1;
        }
    }
```

```

        for (j = i - 1; j >= low; --j)
            r[j + 1] = r[j];          /* 记录依次向后移动，移动次数不变 */

        r[low] = x;                  /* 插入记录 */
    }
}

```

折半插入排序的比较次数少于直接插入排序。插入第 i 个元素时，用折半查找确定位置，比较次数约为 $\log_2 i$ 。但是，确定插入位置后，仍然需要移动元素，因此总时间复杂度仍为 $O(n^2)$ 。

这说明排序算法不能只看“比较次数”。在顺序存储结构中，插入一个元素往往伴随一批元素后移。如果移动代价没有降低，即使查找插入位置更快，整体数量级也可能不变。

折半插入排序是稳定排序。代码中当 `x.key` 与 `r[mid].key` 相等时执行 `low = mid + 1`，这会把新元素插入到已有相等元素之后，从而保持相等关键字的原有次序。

3. 希尔排序

希尔排序又称缩小增量排序，是对直接插入排序的进一步改进。直接插入排序在序列基本有序、规模较小时效率较高；希尔排序正是先通过较大间隔的分组插入，让整个序列快速接近有序，然后逐步缩小间隔，最后进行一次间隔为 1 的插入排序。

教材中的一趟希尔插入排序代码如下：

```

void ShellInsert(RecordType r[], int n, int delta) {
    /* 对记录数组r做一趟希尔插入排序，n为数组的长度，delta为增量 */
    int i, j;

    for (i = 1 + delta; i <= n; i++) {
        /* 1 + delta为第一个子序列的第二个元素的下标 */
        if (r[i].key < r[i - delta].key) {
            r[0] = r[i];          /* 备份r[i]，不做监视哨 */

            for (j = i - delta; j > 0 && r[0].key < r[j].key; j -= delta) {
                r[j + delta] = r[j];
            }

            r[j + delta] = r[0];
        }
    }
}

```

教材中的完整希尔排序代码如下：

```

void ShellSort(RecordType r[], int n, int delta[], int m) {
    /* 对记录数组r做希尔排序，n为数组r的长度，delta为增量数组，m为delta[]的长度 */
    int i;

    for (i = 0; i <= m - 1; ++i) {
        ShellInsert(r, n, delta[i]);
    }
}

```


希尔排序的关键不在某一次插入，而在增量序列。每一趟排序只处理相距 `delta` 的元素。当 `delta` 较大时，元素可以跨越较长距离移动，能够较快减少序列中的逆序；当 `delta` 最终变为 1 时，序列已经基本有序，此时直接插入排序的效率较高。

个人实现中写出了一趟按增量插入的核心：

```
int v = A[i];
int j = i - g;

while (j >= 0 && A[j] > v)
{
    A[j + g] = A[j];
    j -= g; //直插同款
}

A[j + g] = v;
```

希尔排序的时间复杂度与增量序列有关，教材中给出的一般估计为 $O(n^{1.5})$ ，优于直接插入排序的 $O(n^2)$ 。它的空间复杂度为 $O(1)$ 。由于分组插入可能让相等关键字跨组移动，改变原有相对次序，因此希尔排序是不稳定排序。

三、交换类排序

交换类排序通过交换逆序元素来完成排序。冒泡排序和快速排序都属于交换类排序，但二者效率差别很大：冒泡排序每次只交换相邻元素，而快速排序通过一次划分可能消除多个逆序。

1. 冒泡排序

冒泡排序反复扫描待排序序列，比较相邻两个记录的关键字。如果相邻记录逆序，就交换它们的位置。每一趟扫描后，当前未排序部分的最大关键字记录会被放到末尾。

教材中的冒泡排序代码可以整理如下：

```
void BubbleSort(RecordType r[], int n) {
    /* 对记录数组r做冒泡排序，n为数组的长度 */
    int i, j;
    int change;
    RecordType x;

    change = TRUE;

    for (i = 1; i <= n - 1 && change; ++i) {
        change = FALSE;

        for (j = 1; j <= n - i; ++j) {
            if (r[j].key > r[j + 1].key) {
                x = r[j];
                r[j] = r[j + 1];
                r[j + 1] = x;
                change = TRUE;
            }
        }
    }
}
```



```
}
```

这段代码中的 `change` 是冒泡排序的重要优化。如果某一趟扫描没有发生任何交换，说明序列已经有序，可以提前结束。没有这个标记时，即使序列已经排好，算法仍会继续执行剩余趟数。

个人实现中的冒泡排序固定执行 $n - 1$ 趟：

```
for (i = 1; i < 5; i++)
{
    for (j = 0; j < 5 - i; j++)
    {
        if (a[j] < a[j + 1])
        {
            swap(a[j], a[j + 1]);
        }
    }
}
```

这段代码实现的是降序排序，因为判断条件是 `a[j] < a[j + 1]`。如果要升序排序，应改成 `a[j] > a[j + 1]`。与教材相比，它缺少 `change` 提前结束优化。可以改为：

```
for (int i = 1; i < n; i++)
{
    bool change = false;

    for (int j = 0; j < n - i; j++)
    {
        if (a[j] > a[j + 1])
        {
            swap(a[j], a[j + 1]);
            change = true;
        }
    }

    if (!change) break;
}
```

冒泡排序的最好情况是原序列已经有序，加入 `change` 后只需一趟扫描，时间复杂度为 $O(n)$ 。最坏情况是原序列逆序，需要进行约 $n(n - 1)/2$ 次比较，时间复杂度为 $O(n^2)$ 。空间复杂度为 $O(1)$ 。

冒泡排序是稳定排序。因为它只在相邻元素逆序时交换，若两个关键字相等，则不会交换，因此相等元素的相对次序保持不变。

2. 快速排序

<https://xiaomaoyang.top/2026/06/03/quicksort/>

快速排序是对冒泡排序思想的改进。冒泡排序通过相邻交换逐步消除逆序，一次交换通常只能消除一个逆序；快速排序通过选取基准记录，将序列划分为左右两个子表，一次划分可能消除多个逆序，因此平均效率更高。

快速排序的基本思想是：选择一个基准记录 `x`，使基准左侧记录的关键字都小于它，右侧记录的关键字都大于或等于它；然后递归地对左右两个子表继续排序。

教材中的完整快速排序代码如下：

```
void QKSort(RecordType r[], int low, int high) {
    /* 对记录数组r[low..high]用快速排序算法进行排序 */
    int pos;

    if (low < high) {
        pos = QKPass(r, low, high);    /* 以基准记录为界划分两个子表 */
        QKSort(r, low, pos - 1);      /* 对左部子表快速排序 */
        QKSort(r, pos + 1, high);      /* 对右部子表快速排序 */
    }
}
```

教材中的一趟快速排序划分代码如下：

```
int QKPass(RecordType r[], int low, int high) {
    /*
     * 用基准记录对r[low]~r[high]部分进行一趟划分，
     * 使基准记录之前所有记录的关键字小于基准记录，
     * 基准记录之后所有记录的关键字大于或等于基准记录。
     */
    RecordType x;

    x = r[low];    /* 选择基准记录 */

    while (low < high) {
        while (low < high && r[high].key >= x.key)
            high--;    /* high从右到左找小于x.key的记录 */

        if (low < high) {
            r[low] = r[high];
            low++;    /* 将小于x.key的记录送入空单元r[low] */
        }

        while (low < high && r[low].key < x.key)
            low++;    /* low从左到右找大于或等于x.key的记录 */

        if (low < high) {
            r[high] = r[low];
            high--;    /* 将大于或等于x.key的记录送入空单元r[high] */
        }
    }

    r[low] = x;    /* 将基准记录保存到low=high的位置 */
    return low;    /* 返回基准记录的位置 */
}
```

教材代码采用的是“挖坑法”。先把 `r[low]` 保存到变量 `x` 中，使 `r[low]` 成为空单元；再从右侧找小于基准的记录放到左侧空位，从左侧找大于或等于基准的记录放到右侧空位；当 `low` 与 `high` 相遇时，将基准记录放回该位置。

个人实现中采用的是 **双指针**：

```
int x = q[l], i = l - 1, j = r + 1;

while (i < j)
{
    do i++; while (q[i] < x);
    do j--; while (q[j] > x);

    if (i < j) swap(q[i], q[j]);
}

quicksort(q, l, j);
quicksort(q, j + 1, r);
```

这个写法与教材代码的分区方式不同。教材通过不断填补空位完成划分；个人实现通过两个指针向中间扫描，遇到左右两侧不符合要求的元素就直接交换。这个写法的优点是代码短，但边界细节更敏感。

其中 `do...while` 是一个有价值的实现细节。它能保证指针至少移动一次，避免在大量元素等于基准值时，`i` 和 `j` 都停在原地导致死循环。递归区间写成 `[l, j]` 和 `[j + 1, r]`。

快速排序的最好情况是每次划分都接近均分，此时时间复杂度为 $O(n \log n)$ 。平均情况下快速排序也为 $O(n \log n)$ ，并且常数因子较小，实际效率很高。最坏情况是每次基准都选到最大值或最小值，划分极不平衡，时间复杂度退化为 $O(n^2)$ 。

快速排序的空间复杂度来自递归栈。平均情况下递归深度约为 $O(\log n)$ ，因此辅助空间复杂度为 $O(\log n)$ ；最坏情况下递归深度可能达到 $O(n)$ 。

快速排序通常是不稳定排序。因为划分过程中的交换可能让相等关键字跨越彼此，改变原有相对次序。例如相等关键字分布在基准两侧时，交换和递归划分都可能破坏稳定性。

四、范围内算法对比

算法	类别	核心思想	最好时间	平均时间	最坏时间	空间复杂度	稳定性
直接插入排序	插入类	将当前元素插入前面有序区间	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	插入类	用折半查找确定插入位置	比较次数较少	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	插入类	按增量分组插入，逐步缩小增量	与增量有关	常见估计 $O(n^{1.5})$	与增量有关	$O(1)$	不稳定
冒泡排序	交换类	反复交换相邻逆序元素	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定

算法	类别	核心思想	最好时间	平均时间	最坏时间	空间复杂度	稳定性
快速排序	交换类	基准划分后递归处理子表	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	平均 $O(\log n)$	不稳定

从表中可以看出，插入类排序的重点是“把元素放入已有序区间”。直接插入排序实现简单，在基本有序时效率很高；折半插入排序减少了比较次数，但没有减少移动次数；希尔排序通过增量分组使序列快速接近有序，因此通常比直接插入排序更快，但失去了稳定性。

交换类排序的重点是“通过交换消除逆序”。冒泡排序只交换相邻元素，理解简单但效率较低；快速排序通过一次划分消除大量逆序，平均性能明显更好，但需要注意基准选择和递归边界。

五、综合分析

本阶段学习的排序算法可以围绕两条主线理解。

第一条主线是插入思想。直接插入排序从“已有序区间”出发，逐个插入新元素；折半插入排序优化的是查找插入位置的过程；希尔排序优化的是序列整体状态，先通过大步长移动减少逆序，再利用直接插入排序在基本有序时效率高的特点完成最终排序。

第二条主线是交换思想。冒泡排序通过相邻交换逐步把最大值推到末尾，过程直观但效率有限；快速排序通过基准划分把序列分成两个子表，再递归处理，平均复杂度从 $O(n^2)$ 降到 $O(n \log n)$ 。

在实际选择算法时，需要结合数据规模、初始有序程度、稳定性要求和最坏情况风险。若数据规模较小或基本有序，直接插入排序简单有效；若希望在中等规模数据上获得比直接插入更好的效果，可以考虑希尔排序；若追求平均效率，快速排序更有优势；若必须保持稳定性，则直接插入排序、折半插入排序和冒泡排序更合适。